

# GDD: Pickleball Smash — Game Design Document

Version: 1.1

Date: May 31, 2026

Review Status:  Reviewed by reasoning agent — issues applied

Engine: Godot 4 (GDScript)

Platform: Mobile (iOS + Android)

## 1. Game Architecture

### Scene Tree Structure

|                       |                                                         |
|-----------------------|---------------------------------------------------------|
| Game/                 |                                                         |
| ├ Main.tscn           | – Entry point, handles state machine                    |
| ├ Menus/              |                                                         |
| └ MainMenu.tscn       | – Title screen, play/shop/settings                      |
| └ Matchmaking.tscn    | – Finding opponent screen                               |
| └ Shop.tscn           | – Cosmetics and currencies                              |
| └ Collection.tscn     | – Paddles, characters, courts                           |
| └ Settings.tscn       | – Audio, controls, account                              |
| └ Leaderboard.tscn    | – Global + friends leaderboard                          |
| ├ Gameplay/           |                                                         |
| └ Court.tscn          | – Main match scene                                      |
| └ Player.tscn         | – Player character/paddle                               |
| └ Ball.tscn           | – Pickleball (physics object)                           |
| └ Net.tscn            | – Net collision                                         |
| └ Tutorial.tscn       | – Interactive gesture training, first match scripted AI |
| └ UI/                 |                                                         |
| └ HUD.tscn            | – Score, timer, power meter                             |
| └ PauseMenu.tscn      | – In-game pause overlay                                 |
| └ ResultScreen.tscn   | – Post-match results                                    |
| └ PowerIndicator.tscn | – Shot power indicator                                  |
| └ GestureGuide.tscn   | – Swipe gesture visual helper (first 5 matches)         |
| ├ Systems/            |                                                         |
| └ GameState.gd        | – State machine (serve/play/score/pause)                |
| └ Physics.gd          | – Ball physics calculations                             |
| └ MatchManager.gd     | – Match lifecycle, scoring, rules                       |
| └ InputHandler.gd     | – Touch input → shot type mapping                       |
| └ AIManager.gd        | – AI opponent behavior                                  |
| └ AudioManager.gd     | – SFX, BGM, spatial audio for ball contacts             |
| └ EventBus.gd         | – Global signal bus for cross-system communication      |

```
| | └─ AnalyticsManager.gd – Instrumentation for all success metrics (D1/D7/D30, session length, etc.)
| | └─ Network/
| |   └─ NetworkManager.gd – WebSocket/server communication
| |   └─ SyncManager.gd   – Real-time state sync
| └─ Progression/
|   └─ PlayerData.gd      – Save/load player progress
|   └─ RankingSystem.gd   – ELO/MMR calculations
|   └─ EconomyManager.gd  – Currency and purchases
```

## 2. Ball Physics System

### Core Ball Properties

Full 3D physics in `RigidBody3D`. The court is a 3D space with perspective camera — ball height, spin curves, and net clearance are all fully 3D.

```
# Ball.gd
extends RigidBody3D

@export var base_speed: float = 15.0
@export var max_speed: float = 35.0
@export var spin_factor: float = 0.3
@export var linear_drag: float = 0.08 # Pickleball wiffle ball drag – ball slows dramatically

var spin_vector: Vector3 = Vector3.ZERO # Topspin/backspin/sidespin
var shot_type: ShotType = ShotType.DRIVE

enum ShotType { DINK, DRIVE, LOB, VOLLEY, ERNE, ATP }
```

### Pickleball Air Resistance (Wiffle Ball Drag)

Unlike tennis balls, a pickleball is a **plastic wiffle ball with holes** — air resistance is a defining characteristic. The ball slows dramatically mid-flight, which is fundamental to kitchen play and dinking.

#### Drag model (velocity-dependent):

```
# Applied in Ball._integrate_forces()
func _apply_drag(linear_velocity: Vector3, delta: float) -> void:
    var speed: float = linear_velocity.length()
    if speed < 1.0:
        return

    # Quadratic drag:  $F \approx -v * |v| * C_d$ 
    # Pickleball drag coefficient is ~4x higher than a tennis ball
    var drag_coefficient: float = 0.08
```

```
var drag_force: Vector3 = -linear_velocity.normalized() * speed * speed * drag_coefficient
apply_central_force(drag_force)

# Ball slows ~30% over 7.6m court length at full drive speed
# Dinks slow ~50% – making them land softly in the kitchen
```

This drag model creates the authentic pickleball feel:

- Drives arrive faster than dinks but still decelerate significantly
- Dinks barely make it past the kitchen before dying — authentic
- Lobs maintain more speed due to ballistic arc (less horizontal distance affected)

| Shot   | Speed         | Trajectory                | Spin               | Notes                                |
|--------|---------------|---------------------------|--------------------|--------------------------------------|
| Dink   | Slow (5-8)    | Low arc, drops in kitchen | Slight underspin   | Must clear net, must land in kitchen |
| Drive  | Fast (15-25)  | Flat trajectory           | Topspin            | High power, lower control            |
| Lob    | Medium (8-12) | High arc, deep            | Minimal            | Risk of smash return                 |
| Volley | Fast (10-20)  | Sharp angle down          | Sidespin available | Cannot hit from kitchen zone         |
| Erne   | Fast (15-22)  | Extreme angle             | Heavy topspin      | Unlockable, high risk/reward         |
| ATP    | Varied        | Around net post           | Sidespin           | Unlockable, situational              |

## Pickleball-Specific Rules (implemented in MatchManager)

```
# MatchManager.gd – Core rule enforcement
var hits_in_rally: int = 0 # Tracks total hits in current rally

# 1. TWO-BOUNCE RULE
func must_ball_bounce() -> bool:
    # First two hits (serve + return of serve) must bounce
    return hits_in_rally < 2

func record_hit():
    hits_in_rally += 1

func reset_rally():
    hits_in_rally = 0

# 2. KITCHEN ZONE (No Volley Zone)
func check_kitchen_violation(hitter_position: Vector3, ball_on_player_side: bool) -> bool:
    var kitchen_rect = get_kitchen_rect_for_player(ball_on_player_side)
    # Player cannot volley while standing in the kitchen
    return kitchen_rect.has_point(hitter_position) and not must_ball_bounce()

# 3. UNDERHAND SERVE
func validate_serve(swing_direction: String, contact_height: float) -> bool:
    if swing_direction != "upward":
```

```
    return false # Serve must be underhand (upward swing)
    return contact_height < 0.914 # Waist height (36 inches) max
}
```

### 3. Court Layout (3D Perspective Camera)

The court is a full 3D scene. The camera is positioned behind the player at a slight angle (3/4 perspective), showing the full court with depth. Net, kitchen zones, and service areas are all 3D meshes with collision.



#### Dimensions (normalized for mobile):

- Court width: 1.0 (full screen)
- Court height: 1.8 (vertical orientation preferred)
- Net height: 0.5 (center)
- Kitchen depth: 0.2 from net on each side
- Service areas: split left/right behind kitchen

## 4. Controls — Detailed Design

### Serve Mechanic

Serving is a two-step gesture distinct from regular play:

1. **Tap ball button** → Ball appears in player's hand, ready to serve
2. **Swipe upward** → Underhand serve motion. Speed of swipe = serve power. Directional aim by swiping left/right/center
3. **Auto-placement:** Server must be behind baseline. Ball auto-bounces once on opponent's side (two-bounce rule enforced)

#### Serve constraints:

- Contact point must be below waist height (~36in / 0.914m)
- Swing direction must be upward (underhand)
- Ball must clear the net and land in the diagonal service court
- Serve must bounce before opponent can volley

### Touch Input States

IDLE → TOUCH\_DOWN (finger on screen)  
→ SWIPE (finger moved quickly)  
→ DRAG (finger held + moved slowly)  
→ RELEASE (finger lifted)  
→ TAP (short touch, no movement)

### Control Mapping

| Gesture          | Detection                                    | Shot Result                       |
|------------------|----------------------------------------------|-----------------------------------|
| Swipe Up         | velocity.y < -threshold, distance > min      | Lob — ball arcs high and deep     |
| Swipe Down       | velocity.y > threshold, distance > min       | Dink — soft shot into kitchen     |
| Swipe Left       | velocity.x < -threshold, distance > min      | Cross-court to left side          |
| Swipe Right      | velocity.x > threshold, distance > min       | Cross-court to right side         |
| Tap              | touch_duration < 0.15s, distance < threshold | Volley (if ball on player's side) |
| Tap + Drag Hold  | hold 0.3s + drag direction                   | Aimed precision shot              |
| Quick Double Tap | 2 taps within 0.3s                           | Power shot (hold to charge meter) |

### Player Positioning (Auto-Move)

The player character automatically moves into position based on input. The player controls **where to aim**, not **where to run** — similar to Tennis Clash.

- On Swipe Up (Lob):
  - AI moves player back
  - ↳ Ball arcs high to opponent's baseline
- On Swipe Down (Dink):
  - AI moves player forward toward kitchen
  - ↳ Ball drops softly into opponent's kitchen

Advanced players can override auto-position by tapping a position on the court before the ball crosses.

## 5. AI Behavior

### Difficulty Levels

| Level    | Reaction Time | Shot Accuracy | Strategic Play               |
|----------|---------------|---------------|------------------------------|
| Beginner | 0.8s          | 60%           | Hits returns only            |
| Casual   | 0.6s          | 70%           | Basic shot selection         |
| Pro      | 0.4s          | 82%           | Mixes dinks and drives       |
| Elite    | 0.25s         | 92%           | Kitchen play, sets up points |
| Champion | 0.15s         | 96%           | Full strategy, erne attempts |

### AI Decision Tree

- Ball incoming →
  - Can reach ball? → No → Out of position, late return (weak)

→ Yes →
  - Is ball in kitchen? → Yes → Dink or let bounce

→ No →
  - Is opponent at net? → Yes → Lob over them

→ No →
  - Player position → Deep → Dink to bring them forward

→ Mid → Drive cross-court

→ Kitchen → Lob to reset
  - Mix in random shot variation (10-15% non-optimal for realism)

## 6. Progression Systems

### Player XP Table

| Level | XP Required | Unlock                     |
|-------|-------------|----------------------------|
| 1     | 0           | Quick Match, Practice      |
| 3     | 200         | Ranked Mode                |
| 5     | 500         | Shop                       |
| 7     | 900         | Doubles Mode               |
| 10    | 1800        | Events                     |
| 15    | 3500        | Tournaments                |
| 20    | 6000        | Season Pass                |
| 10    | 1800        | Erne Shot (advanced skill) |
| 12    | 2500        | ATP Shot (advanced skill)  |

### Ranked Tiers

| Tier           | MMR Range | Promotion       | Demotion                 |
|----------------|-----------|-----------------|--------------------------|
| Bronze I-III   | 0-599     | Win 3 matches   | -                        |
| Silver I-III   | 600-1199  | Win 3 matches   | Losing streak protection |
| Gold I-III     | 1200-1999 | Win 4 matches   | Lose 3 in a row          |
| Platinum I-III | 2000-2999 | Win 5 matches   | Lose 2 in a row          |
| Diamond I-III  | 3000-4499 | Win 5 matches   | Lose 2 in a row          |
| Elite          | 4500-5999 | Win 6 matches   | Lose 2 in a row          |
| Champion       | 6000+     | Top 500 players | Occupy slot              |

### Season Rewards

- **Bronze:** Participation badge, 50 coins
- **Silver:** Silver border, 100 coins, basic skin
- **Gold:** Gold border, 250 coins, exclusive gold paddle skin
- **Platinum:** Platinum trail effect, 500 coins, exclusive skin
- **Diamond:** Diamond trail + border, 1000 coins, legendary skin
- **Elite:** Elite title, 2000 coins, animated legendary skin
- **Champion:** Champion title + global leaderboard display, 5000 coins, exclusive champion paddle

## 7. Monerization Details

### Soft Currency — Coins

- Earned: 15-50 per match (based on performance)
- Daily bonus: 100-200 coins
- Uses: Paddle upgrades (200-2000 coins), basic cosmetics

### Hard Currency — Gems

- Earned: Ranked milestones, first-win daily bonus
- Purchase: \$0.99 (50) to \$49.99 (3000)
- Uses: Season pass (500 gems), premium cosmetics, event entry

### Season Pass (Monthly)

- Free track: Basic rewards for all players
- Premium track (\$4.99): Exclusive skins, paddles, cosmetics, bonus gems
- Elite track (\$9.99): Premium + instant 10 levels + exclusive emote

## 8. Art & Audio Style

### Visual Direction

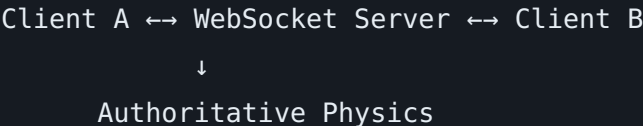
- **Theme:** Bright, colorful, clean — casual sport aesthetic
- **Style:** Stylized realism (not hyper-realistic, not cartoon)
- **Color Palette:** Court blue/green backgrounds, white lines, neon accent on power shots
- **Characters:** Simple humanoid figures with visible paddle and outfit, focus on court visibility

### Audio

- **BGM:** Upbeat, energetic instrumental (different per court theme)
- **SFX:** Paddle contact (wooden thwack), ball bounce (hollow pock), net hit, crowd ambience
- **UI Sounds:** Menu clicks, score updates, rank-up fanfare

## 9. Network & Multiplayer

### Architecture





(Server validates shots,  
prevents cheating)

## Match Flow

1. **Matchmaking:** Player selects mode → ELO-based search (30s max) → Found!
2. **Sync:** Both clients load court → Start signal received
3. **Gameplay:** Client sends inputs → Server validates → Broadcasts ball state (20-30 updates/sec with client-side prediction and position interpolation)
4. **Reconnection:** If a client disconnects, a 30-second timer starts. AI takes over the disconnected player. Client can rejoin within the window
5. **Resolution:** Game ends → Server declares winner → Stats recorded

## Anti-Cheat Measures

- Server validates all shot trajectories and timings
- Speed caps on ball (client can't send speed > max)
- Position reconciliation every 10 frames
- Rate limiting on inputs

---

## 10. Godot 4 Implementation Notes

---

### Physics: Full 3D

The game uses **full 3D physics (RigidBody3D)** with a 3D perspective camera. Ball height is real (y-axis), spin affects trajectory in all 3 dimensions, and net clearance is physically accurate.

### Key Nodes & Components

- `RigidBody3D` for ball physics with custom `_integrate_forces` for drag modeling
- `CharacterBody3D` for player with kinematic motion
- `NavigationAgent3D` for AI pathfinding to ball positions
- `AnimationPlayer` for paddle swing/character celebrations
- `MultiplayerSynchronizer` for state synchronization
- `Resource` files for paddle/character stat configurations

### Performance Optimizations

- Use `NavigationAgent3D` for AI pathfinding to ball positions
- Object pooling for trail particles and ball effects
- LOD (Level of Detail) for distant characters
- Texture atlas for all UI elements
- Mobile-specific: Reduce shadow quality, use mobile-friendly shaders

## Touch Input

- Use `InputEventScreenTouch` and `InputEventScreenDrag`
- Implement gesture recognizer in `InputHandler.gd`
- Support for simultaneous touch (drag to aim + release to hit)

## 11. Future Roadmap (Post-Launch)

| Update | Content                                              |
|--------|------------------------------------------------------|
| v1.1   | New court themes, 2 more characters                  |
| v1.2   | Clan/guild system, private lobbies                   |
| v1.3   | Replay system, spectate mode                         |
| v2.0   | Tournament mode, 2v2 ranked                          |
| v2.1   | Custom matches (change rules, court size)            |
| v3.0   | Season 2: New shot types, new characters, event pass |